

## 《计算机导论》第五部分：数据结构、算法与程序设计 - 知识点...

### 《计算机导论》第五部分：数据结构、算法与程序设计 - 知识点详解与复习指导

#### 一、主要内容概述

本部分系统性地讲解了计算机科学的核心基础：**数据结构**、**算法与程序设计**。这三者是计算机科学解决问题、实现软件功能的基石。

#### 5.1 什么是数据结构

#### 5.2 几种典型的数据逻辑结构

#### 5.3 什么是算法

#### 5.4 算法的描述工具和设计举例

#### 5.5 算法的评价

#### 5.6 程序设计语言的发展历程

#### 5.7 良好的程序设计习惯

#### 5.8 软件和硬件视点审视计算机的工作过程

1. **数据结构**：研究如何组织、存储和管理数据，以便高效地访问和修改。
2. **算法**：一系列明确的、有限的步骤，用于解决特定问题或执行特定计算。
3. **程序设计**：使用程序设计语言将算法转化为计算机可执行的指令，涉及到语言的发展、编程范式以及良好的编程习惯。

#### 二、知识点详述

##### 5.1 什么是数据结构

- **数据：**描述客观事物的数、字符以及所有能输入到计算机中并被计算机程序处理的符号集合。数据是信息的载体。
- **数据结构：**
  - **定义：**带有结构的数据元素集合，反映数据元素相互之间存在的某种关系。
  - **视角：**
    - i. **计算机学科视角：**研究数据的逻辑结构、物理结构及其关系。
    - ii. **课程视角：**计算机专业的核心基础课，是设计操作系统、编译系统等大型软件的基础。
  - **重要性：**是算法的基石，好的数据结构能使问题解决更简单、高效。
- **数据类型：**
  - **定义：**一个值的集合以及定义在这个值集上的一组操作。它决定了数据的存储方式和允许的操作。
  - **基本数据类型：**
    - **Boolean：**逻辑值 (True/False)。
    - **Byte：**字节 (0-255)。
    - **Char：**字符。
    - **DateTime：**日期时间。
    - **数值类型：**Decimal (精确小数)、Double (双精度浮点)、Single (单精度浮点)、Integer (整型)、Long (长整型)、Short (短整型) 及其无符号版本 (UInteger, ULong, UShort)。
    - **String：**字符串。
  - **复合数据类型：**
    - **记录/结构体 (Record/Struct)：**包含不同类型数据项的结构 (如学生成绩单)。
    - **数组 (Array)：**由相同类型数据元素组成的集合，通过索引访问。
    - **对象 (Object)：**面向对象编程的核心概念。将一组属性和操作封装在一起。
    - **类 (Class)：**对具有相同或相似性质对象的抽象描述和定义。对象是类的实例。
    - **面向对象核心概念：**
      - **状态：**对象用数据值描述的状态。

- **行为**：对象可以执行的操作。
- **封装**：将数据和操作结合在对象统一体中。
- **继承**：子类继承父类的属性和方法。
- **多态**：同一操作作用于不同对象产生不同行为。

## 5.2 几种典型的数据逻辑结构

- **逻辑结构**：数据元素之间的逻辑关系（组织形式）。
  - **前趋**：某元素的直接前一个元素。
  - **后继**：某元素的直接后一个元素。
- **主要逻辑结构类型**：
  - a. **线性结构**：数据元素间是一对一的前趋后继关系。
    - **线性表 (Linear List)**：有限个相同类型数据元素组成的有序序列。基本操作：置空、求长、取元素、查找、插入、删除。
    - **栈 (Stack)**：插入和删除操作仅在一端（栈顶）进行的线性表。特性：**后进先出 (LIFO)**。用于递归、函数调用等。
    - **队列 (Queue)**：只允许在一端（队尾）插入，在另一端（队头）删除的线性表。特性：**先进先出 (FIFO)**。用于任务调度、缓冲等。
  - b. **树形结构**：非线性结构，元素间存在一对多的层次关系。
    - **定义**：有且仅有一个根结点（无前趋），其他结点有且仅有一个直接前趋，但可以有多个或多个直接后继。
    - **基本术语**：
      - **结点**：包含数据项及指向其他结点的分支。
      - **度**：结点拥有的子树数。树的度是树中所有结点的度的最大值。
      - **叶子结点**：度为0的结点。
      - **分支结点**：度不为0的结点。
      - **父/子结点、兄弟结点**。
      - **层次与深度**：根为第1层，树中结点的最大层次为树的深度。
    - **二叉树遍历**：
      - **先序遍历**：根 -> 左子树 -> 右子树。

- **中序遍历**：左子树 -> 根 -> 右子树。
- **后序遍历**：左子树 -> 右子树 -> 根。
- c. **图结构**：最一般的非线性结构，结点间是多对多的关系，任意两个结点都可能相邻。
  - **定义**：Graph=(V, E)，V是顶点集，E是边集。
  - **分类**：
    - **无向图**：边无方向。
    - **有向图**：边有方向。
    - **带权图/网络**：边被赋予权重。
- **物理/存储结构**：逻辑结构在计算机内存中的实现方式。
  - a. **顺序存储**：元素存放在地址连续的存储单元中。逻辑相邻则物理相邻。优点：存储密度大，随机存取快。缺点：插入删除需移动大量元素。
  - b. **链式存储**：元素存储在任意的存储单元中，通过指针表示元素间的逻辑关系。优点：插入删除灵活。缺点：存储密度低，访问需遍历。
  - c. **索引存储**：建立索引表记录元素的存储地址。优点：查找快。缺点：需额外存储索引表。
  - d. **散列存储**：根据关键字通过散列函数直接计算存储地址。优点：理想情况下存取极快。缺点：存在冲突问题，需处理。
- **数据处理过程**：选择逻辑结构 -> 确定物理结构 -> 设计基于该结构的运算。

### 5.3 什么是算法

- **定义**：由基本运算及规定的运算顺序所构成的完整解题步骤。是对计算机将输入转化为输出的具体描述。
- **特性**：
  - a. **正确性**：对任何合法输入都能得到预期输出。
  - b. **确定性**：每一步骤含义明确，无二义性。
  - c. **有穷性**：在有限步骤后必然结束。
  - d. **通用性**：解决一类问题，而非个例。
- **计算机求解问题步骤**：
  - a. **问题分析**：定义问题，确定输入/输出，评估可行性。
  - b. **算法设计**：采用“自顶向下，逐步求精”的方法设计求解步骤。

- c. **程序编制**：用程序设计语言实现算法。
- d. **测试**：用测试用例验证程序的正确性。

## 5.4 算法的描述工具和设计举例

- **描述工具**：
  - a. **自然语言**：易懂，但不精确，易产生歧义。
  - b. **流程图**：用图形符号描述算法流程，直观清晰。
  - c. **伪代码**：介于自然语言和编程语言之间的描述方式。结构清晰，接近程序，便于转化为代码。
- **伪代码要素**：
  - **变量与常量**。
  - **赋值语句** (变量 = 表达式)。
  - **表达式**：算术、关系、逻辑表达式及其优先级。
  - **选择/分支语句** (if...then...else, case)。
  - **循环语句** (while, do...until, for)。
  - **函数与过程**：用于模块化。
  - **注释** (//)。
  - **格式**：通过缩进体现层次结构。
- **设计举例**：文档中给出了求和、斐波那契数列、三角形面积、最大公约数（欧几里得算法）等经典算法的伪代码描述。

## 5.5 算法的评价

- **评价标准**：
  - a. **时间复杂性 (Time Complexity)**：执行算法所需的时间开销，通常表示为问题规模 $n$ 的函数  $T(n)$ 。常用**大O表示法** ( $O(f(n))$ ) 来描述增长量级（如  $O(1)$ ,  $O(n)$ ,  $O(n^2)$ ,  $O(\log n)$ ）。
  - b. **空间复杂性 (Space Complexity)**：算法运行所占用的存储空间，表示为问题规模 $n$ 的函数  $S(n)$ 。
  - c. **易读性**：算法结构清晰、易于理解、修改和维护。在现代硬件资源充裕的背景下尤为重要。

- **问题求解思路对比：**以计算1到n的和为例，展示了从低级循环 ( $O(n)$ )、递归 ( $O(n)$ ) 到直接使用数学公式 ( $O(1)$ ) 的不同算法，体现了算法优劣对效率的影响。

## 5.6 程序设计语言的发展历程

- **定义：**人们为了描述计算过程而设计的一种具有语法语义描述的记号。
- **发展阶段：**
  - a. **机器语言：**由0和1组成的二进制指令。直接面向机器，执行效率高，但难编写、难阅读、难调试、不可移植。
  - b. **汇编语言：**用助记符代替机器指令。相比机器语言易读写，但仍面向硬件，与机器语言一一对应，需汇编程序翻译。
  - c. **高级语言：**接近自然语言和数学公式，独立于机器硬件。需编译或解释执行。经历了以下范式演变：
    - **非结构化编程：**早期，大量使用GOTO语句，导致程序结构混乱（“面条代码”）。
    - **结构化编程：**核心是限制或避免使用GOTO，采用顺序、选择、循环三种基本控制结构，以及自顶向下、逐步求精、模块化的设计方法。代表语言：Pascal, C。
    - **面向对象编程 (OOP)：**以对象为核心，将数据和对数据的操作封装在一起。核心概念：类、对象、封装、继承、多态。提高了代码的可重用性、灵活性和可扩展性。代表语言：C++, Java, C#, Python。
- **重要语言：**FORTRAN（科学计算）、COBOL（商业处理）、LISP（人工智能）、ALGOL（算法语言，影响深远）、Pascal（教学与结构化）、C（系统编程）、C++/Java（面向对象）。
- **翻译系统：**
  - **编译程序 (Compiler)：**将整个源程序一次性翻译成目标机器代码，再执行。执行效率高。过程：词法分析 -> 语法分析 -> （中间代码生成 -> 优化） -> 目标代码生成。
  - **解释程序 (Interpreter)：**逐条翻译并立即执行源程序语句。易于调试和移植，但执行效率较低。

## 5.7 良好的程序设计习惯

1. **标识符命名：**含义清晰，风格统一，避免歧义和系统保留字。
2. **表达式书写：**适当使用括号明确优先级；条件判断清晰；封装重复计算。
3. **函数与过程设计：**功能独立，规模适中；多用局部变量，少用全局变量以降低耦合。
4. **代码编排：**使用统一的缩进格式（如2/4个空格或Tab），使程序结构层次分明。
5. **注释：**在程序/模块开头、复杂逻辑处添加必要注释，说明功能、思路和注意事项。

5.8 软件和硬件视点审视计算机的工作过程

（注：文档内容未详细展开此小节，核心思想是从软件（指令序列）和硬件（CPU、内存等部件协作）两个角度理解计算机如何协同工作以执行程序。）

三、 缩略语表

| 缩略语     | 英文全称                                              | 中文全称/释义                |
|---------|---------------------------------------------------|------------------------|
| LIFO    | Last In, First Out                                | 后进先出（栈的特性）             |
| FIFO    | First In, First Out                               | 先进先出（队列的特性）            |
| OOP     | Object-Oriented Programming                       | 面向对象程序设计               |
| BNF     | Backus-Naur Form                                  | 巴科斯-瑙尔范式（描述语法）         |
| API     | Application Programming Interface                 | 应用程序编程接口               |
| CAD     | Computer-Aided Design                             | 计算机辅助设计                |
| ENIAC   | Electronic Numerical Integrator And Computer      | 电子数字积分计算机（第一台通用电子计算机）  |
| EDSAC   | Electronic Delay Storage Automatic Calculator     | 电子延迟存储自动计算机（早期存储程序计算机） |
| FORTRAN | FORmula TRANslation                               | 公式翻译语言                 |
| COBOL   | COMmon Business-Oriented Language                 | 面向商业的通用语言              |
| ALGOL   | ALGOrithmic Language                              | 算法语言                   |
| LISP    | LISt Processing                                   | 表处理语言                  |
| BASIC   | Beginner ‘s All-purpose Symbolic Instruction Code | 初学者通用符号指令代码            |
| O(n)    | Order of n (Big O Notation)                       | (时间复杂度) 阶为n            |

## 四、复习题与答案详解

### 一、选择题

1. 以下关于数据结构的描述，**错误**的是：A. 数据结构包括逻辑结构和物理结构。B. 算法完全独立于数据结构，可以随意设计。C. 好的数据结构可以显著提高算法的效率。D. 线性表、树和图是典型的数据逻辑结构。**答案：B 详解：**算法和数据结构紧密相关。数据结构是算法的操作对象，算法的设计往往依赖于所选的数据结构。一个高效的算法通常建立在合适的数据结构之上。两者不能完全分离。
2. 栈和队列的共同点是：A. 都是先进先出的线性结构。B. 都是后进先出的线性结构。C. 只允许在端点处插入和删除元素。D. 没有共同点。**答案：C 详解：**栈只允许在栈顶进行插入（入栈）和删除（出栈）；队列只允许在队尾插入（入队），在队头删除（出队）。两者都是操作受限的线性表，且操作都被限定在“端点”。A是队列的特点，B是栈的特点。
3. 在二叉树的中序遍历序列中，根结点的位置是：A. 在序列的开头。B. 在序列的结尾。C. 在序列的中间。D. 左子树遍历完之后，右子树遍历之前。**答案：D 详解：**中序遍历的顺序是“左子树 -> 根结点 -> 右子树”。因此，根结点必然出现在其左子树所有结点之后，右子树所有结点之前，即位于遍历序列的“中间”位置（但不一定是几何中心）。
4. 算法的“有穷性”是指：A. 算法程序的长度是有限的。B. 算法只能被有限的用户使用。C. 算法在任何情况下都会在有限时间内结束。D. 算法在任何情况下都会在有限步骤内结束。**答案：D 详解：**有穷性是算法的基本特性，指算法必须由有限的步骤组成，并且每个步骤都在可接受的时间内完成，从而保证算法总会终止。C选项强调“时间”，而算法步骤的执行时间可能因输入或硬件而异，但“步骤”本身必须是有限的。
5. 将高级语言源程序转换为机器可执行代码的系统软件是：A. 汇编程序 B. 编译程序 C. 解释程序 D. 数据库管理系统**答案：B 详解：**编译程序（编译器）负责将用高级语言编写的整个源程序一次性翻译成等价的机器语言目标程序，然后执行该目标程序。汇编程序翻译的是汇编语言。解释程序是边翻译边执行。

### 二、简答题

1. 简述顺序存储结构和链式存储结构的优缺点。**答案：**
  - **顺序存储结构：**
    - **优点：**存储密度高，逻辑上相邻的元素物理上也相邻；可以随机存取任意元素，访问速度快。
    - **缺点：**插入和删除操作需要移动大量元素，效率低；需要预先分配连续的存储空间，可能造成空间浪费或溢出；不易动态扩充。
  - **链式存储结构：**



- **优点：**插入和删除操作灵活，只需修改指针，无需移动元素；存储空间动态分配，利用率高，易于扩充。
- **缺点：**存储密度较低，因为每个结点都需要额外空间存放指针；不支持随机存取，访问元素需要从头遍历，查找效率较低。

2. 什么是面向对象程序设计中的“封装”？它有什么好处？**答案：**

- **封装**是指将对象的属性（数据）和操作（方法）结合成一个独立的单元（即对象），并尽可能隐藏对象的内部实现细节，仅对外提供公开的访问接口。
- **好处：**
  - i. **安全性：**防止外部代码随意修改对象内部数据，保证了数据的完整性和安全性。
  - ii. **简化性：**使用者只需了解对象提供的接口（方法），无需知道内部复杂的实现逻辑，降低了使用复杂度。
  - iii. **可维护性：**内部实现的修改只要不改变对外接口，就不会影响使用该对象的其他代码，提高了代码的可维护性。
  - iv. **模块化：**促进了代码的模块化，使得系统结构更清晰。

3. 请写出计算整数n的阶乘  $n!$  (即  $1*2*3*\dots*n$ ) 的递归函数伪代码。**答案：**

Plain Text

```
1 // 函数名: Factorial, 参数: n (非负整数)
2 Function Factorial(n)
3     If n == 0 OR n == 1 Then
4         Return 1 // 0! 和 1! 都等于 1, 递归基础情形
5     Else
6         Return n * Factorial(n - 1) // 递归步骤:  $n! = n * (n-1)!$ 
7     End If
8 End Function
```